

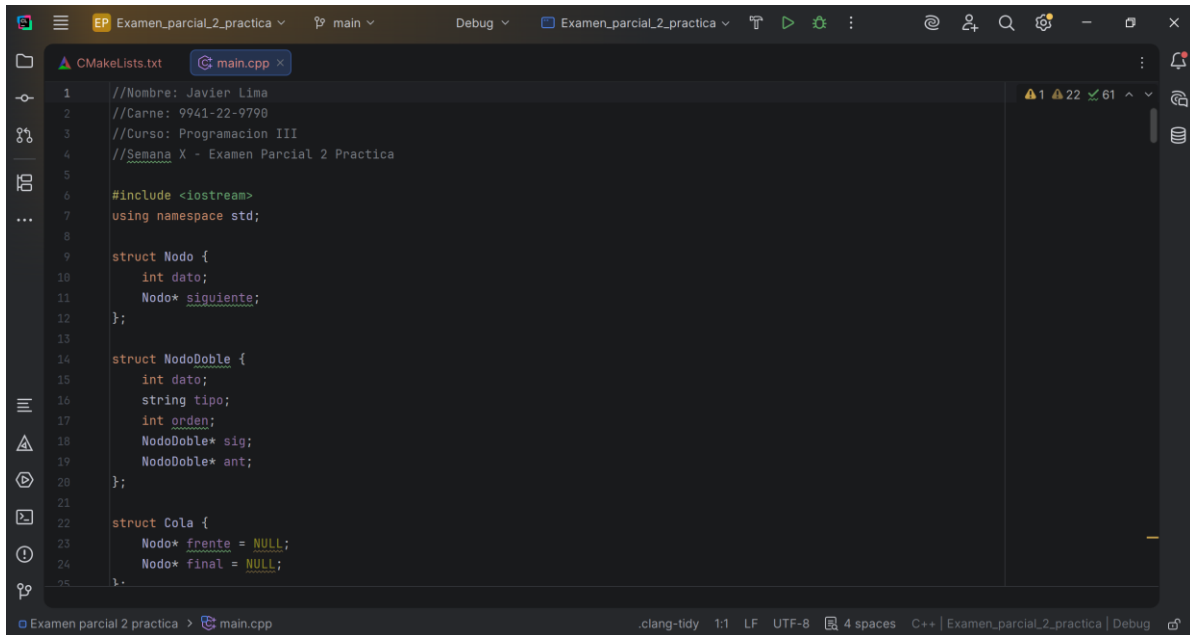


Universidad Mariano Galvez de Guatemala
Facultad de Ingeniería, Matemáticas y Ciencias Físicas
Ingeniería en Sistemas
Programación III
Catedrático: Ing. David Alvarez

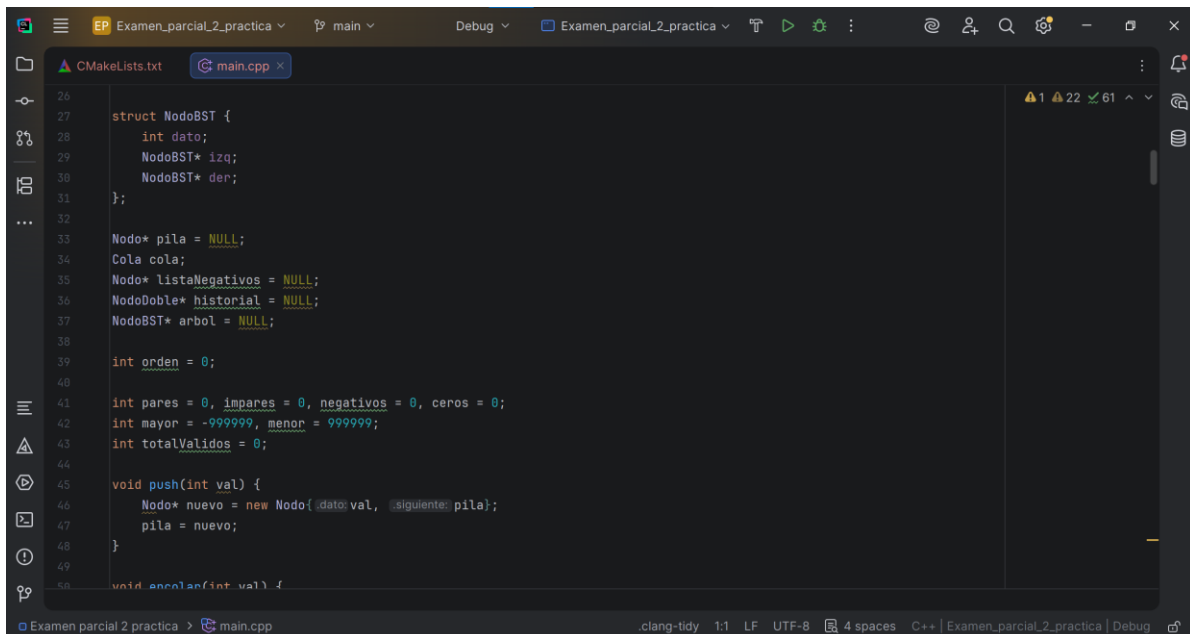
Examen parcial II Parte Practica

Jeremy Javier Lima Guitron
9941-22-9790
25/04/2026

CODIGO



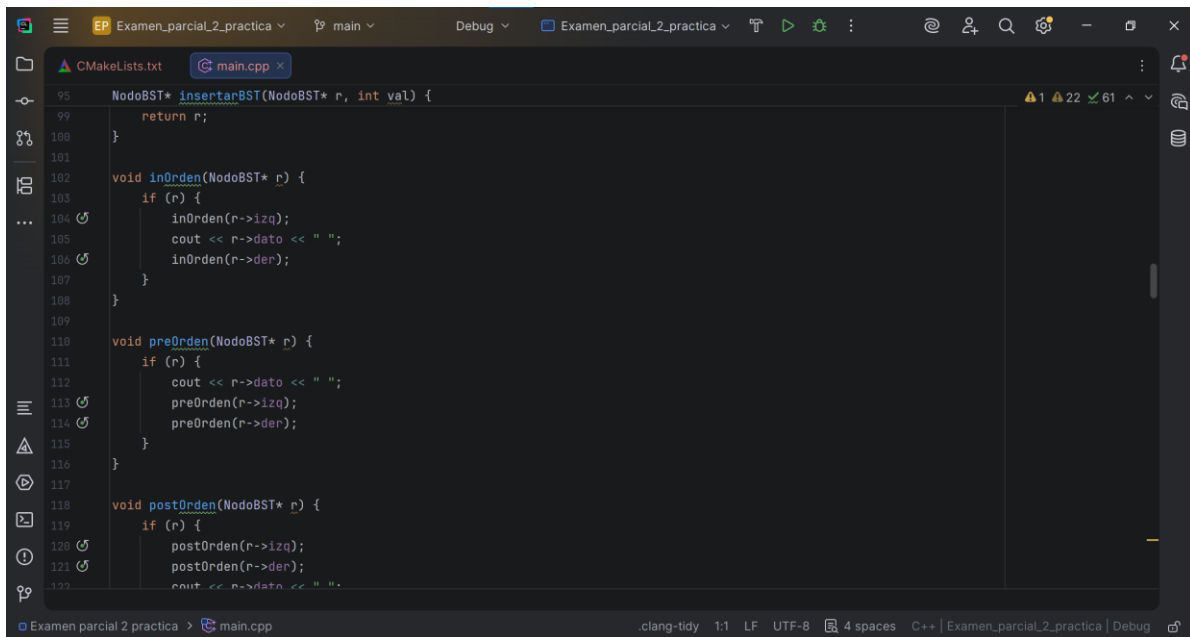
```
1 //Nombre: Javier Lima
2 //Carne: 9941-22-9798
3 //Curso: Programacion III
4 //Semana X - Examen Parcial 2 Practica
5
6 #include <iostream>
7 using namespace std;
8
9 struct Nodo {
10     int dato;
11     Nodo* siguientes;
12 };
13
14 struct NodoDoble {
15     int dato;
16     string tipo;
17     int orden;
18     NodoDoble* sig;
19     NodoDoble* ant;
20 };
21
22 struct Cola {
23     Nodo* frente = NULL;
24     Nodo* final = NULL;
25 }
```



```
26
27 struct NodoBST {
28     int dato;
29     NodoBST* izq;
30     NodoBST* der;
31 };
32
33 Nodo* pila = NULL;
34 Cola cola;
35 Nodo* listaNegativos = NULL;
36 NodoDoble* historial = NULL;
37 NodoBST* arbol = NULL;
38
39 int orden = 0;
40
41 int pares = 0, impares = 0, negativos = 0, ceros = 0;
42 int mayor = -999999, menor = 999999;
43 int totalValidos = 0;
44
45 void push(int val) {
46     Nodo* nuevo = new Nodo{.dato= val, .siguiente= pila};
47     pila = nuevo;
48 }
49
50 void encolar(int val) {
```

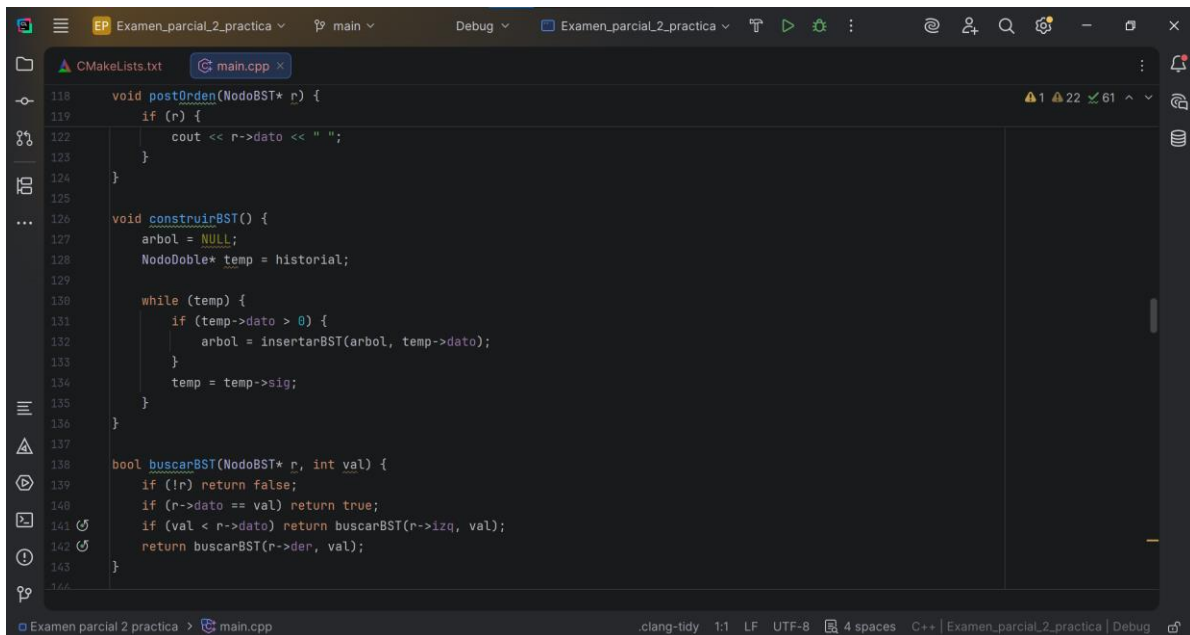
```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
50 void encolar(int val) {
51     Nodo* nuevo = new Nodo{.dato: val, .siguiente: NULL};
52     if (!cola.frente) cola.frente = cola.final = nuevo;
53     else {
54         cola.final->siguiente = nuevo;
55         cola.final = nuevo;
56     }
57 }
58
59 void insertarNegativo(int val) {
60     Nodo* nuevo = new Nodo{.dato: val, .siguiente: listaNegativos};
61     listaNegativos = nuevo;
62 }
63
64 void insertarHistorial(int val, string tipo) {
65     NodoDoble* nuevo = new NodoDoble{.dato: val, .tipo: tipo, ++orden, .sig: NULL, .ant: NULL};
66     if (!historial) historial = nuevo;
67     else {
68         NodoDoble* temp = historial;
69         while (temp->sig) temp = temp->sig;
70         temp->sig = nuevo;
71         nuevo->ant = temp;
72     }
73 }
74
Examen parcial 2 practica main.cpp clang-tidy 1:1 LF UTF-8 4 spaces C++ Examen_parcialL2_practica Debug
```

```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
75 void mostrar(Nodo* lista) {
76     if (!lista) {
77         cout << "Vacía\n";
78         return;
79     }
80     while (lista) {
81         cout << lista->dato << " ";
82         lista = lista->siguiente;
83     }
84     cout << endl;
85 }
86
87 void mostrarHistorial() {
88     NodoDoble* temp = historial;
89     while (temp) {
90         cout << temp->orden << " " << temp->dato << " -> " << temp->tipo << endl;
91         temp = temp->sig;
92     }
93 }
94
95 NodoBST* insertarBST(NodoBST* r, int val) {
96     if (!r) return new NodoBST{.dato: val, .izq: NULL, .der: NULL};
97     if (val < r->dato) r->izq = insertarBST(r->izq, val);
98     else r->der = insertarBST(r->der, val);
99     return r;
100
Examen parcial 2 practica main.cpp clang-tidy 1:1 LF UTF-8 4 spaces C++ Examen_parcialL2_practica Debug
```



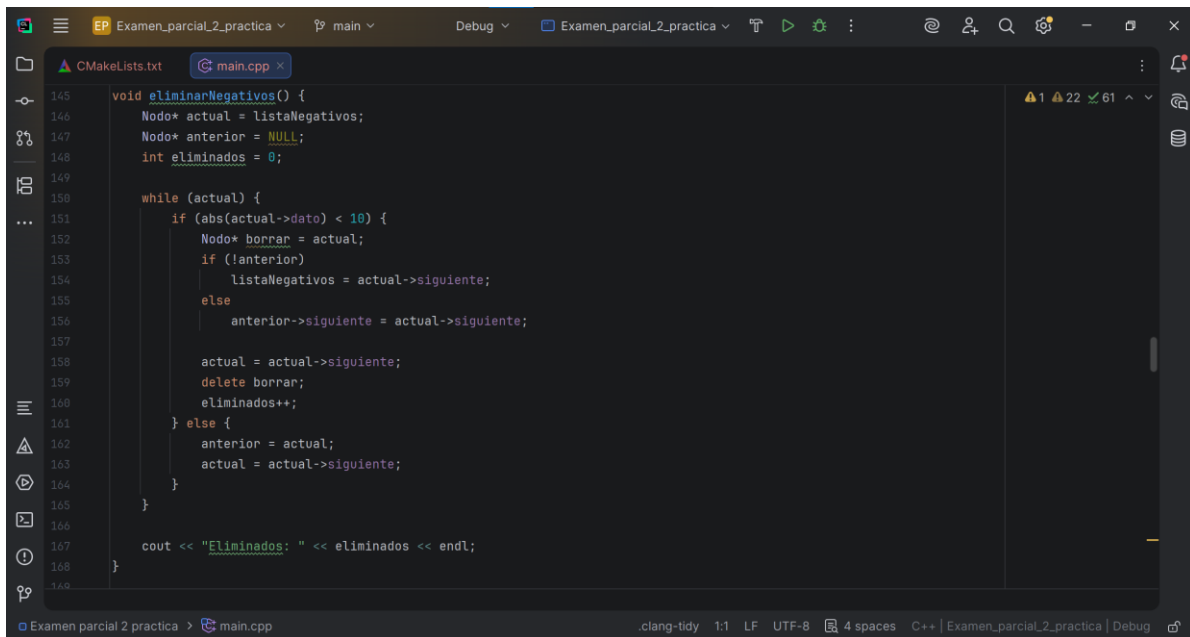
This screenshot shows a C++ IDE with the file 'main.cpp' open. The code defines a binary search tree structure with a 'NodoBST' struct and several functions. The 'insertarBST' function is at line 95, and it returns a pointer to the root of the tree. Below it, there are three traversal functions: 'inOrden' (line 102), 'preOrden' (line 116), and 'postOrden' (line 118). Each traversal function uses a recursive approach to visit the nodes in the specified order. The 'inOrden' function visits the left child, then the node, and then the right child. The 'preOrden' function visits the node first, then the left child, and then the right child. The 'postOrden' function visits the left child, then the right child, and then the node. The code is written in C++ and uses standard library functions like 'cout' for output. The IDE interface includes a sidebar with file explorer, a top bar with project settings, and a bottom status bar showing the current file and compiler options.

```
95  NodoBST* insertarBST(NodoBST* r, int val) {
96      return r;
97  }
98
99  void inOrden(NodoBST* r) {
100      if (r) {
101          inOrden(r->izq);
102          cout << r->dato << " ";
103          inOrden(r->der);
104      }
105  }
106
107  void preOrden(NodoBST* r) {
108      if (r) {
109          cout << r->dato << " ";
110          preOrden(r->izq);
111          preOrden(r->der);
112      }
113  }
114
115  void postOrden(NodoBST* r) {
116      if (r) {
117          postOrden(r->izq);
118          postOrden(r->der);
119          cout << r->dato << " ";
120      }
121  }
```

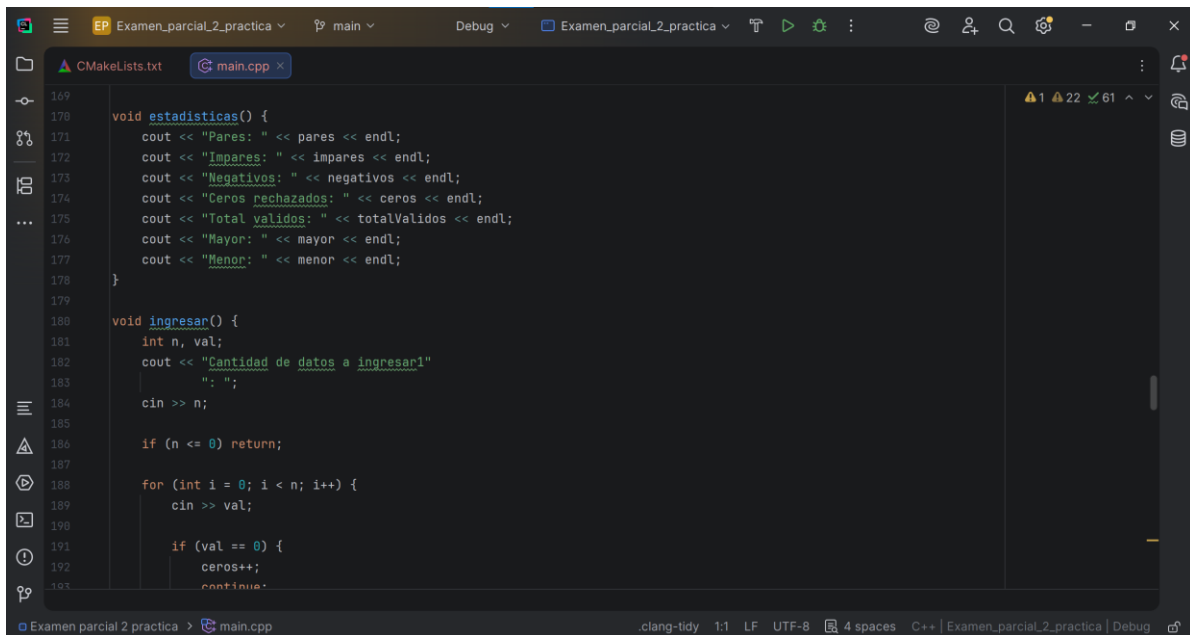


This screenshot shows the same C++ IDE with the file 'main.cpp' open, displaying the 'construirBST' and 'buscarBST' functions. The 'construirBST' function (line 126) is a recursive function that builds a binary search tree from a sequence of values. It uses a 'while' loop to traverse the tree and insert new nodes. The 'buscarBST' function (line 138) is a recursive function that searches for a specific value in the tree. It returns a boolean value indicating whether the value was found. The code is written in C++ and uses standard library functions like 'cout' for output. The IDE interface includes a sidebar with file explorer, a top bar with project settings, and a bottom status bar showing the current file and compiler options.

```
118  void postOrden(NodoBST* r) {
119      if (r) {
120          cout << r->dato << " ";
121      }
122  }
123
124  void construirBST() {
125      arbol = NULL;
126      NodoDoble* temp = historial;
127
128      while (temp) {
129          if (temp->dato > 0) {
130              arbol = insertarBST(arbol, temp->dato);
131          }
132          temp = temp->sig;
133      }
134  }
135
136  bool buscarBST(NodoBST* r, int val) {
137      if (!r) return false;
138      if (r->dato == val) return true;
139      if (val < r->dato) return buscarBST(r->izq, val);
140      return buscarBST(r->der, val);
141  }
```



```
145 void eliminarNegativos() {
146     Nodo* actual = listaNegativos;
147     Nodo* anterior = NULL;
148     int eliminados = 0;
149
150     while (actual) {
151         if (abs(actual->dato) < 10) {
152             Nodo* borrar = actual;
153             if (!anterior)
154                 listaNegativos = actual->siguiente;
155             else
156                 anterior->siguiente = actual->siguiente;
157
158             actual = actual->siguiente;
159             delete borrar;
160             eliminados++;
161         } else {
162             anterior = actual;
163             actual = actual->siguiente;
164         }
165     }
166
167     cout << "Eliminados: " << eliminados << endl;
168 }
```



```
169
170 void estadisticas() {
171     cout << "Pares: " << pares << endl;
172     cout << "Impares: " << impares << endl;
173     cout << "Negativos: " << negativos << endl;
174     cout << "Ceros rechazados: " << ceros << endl;
175     cout << "Total validos: " << totalValidos << endl;
176     cout << "Mayor: " << mayor << endl;
177     cout << "Menor: " << menor << endl;
178 }
179
180 void ingresar() {
181     int n, val;
182     cout << "Cantidad de datos a ingresar:"
183           << endl;
184     cin >> n;
185
186     if (n <= 0) return;
187
188     for (int i = 0; i < n; i++) {
189         cin >> val;
190
191         if (val == 0) {
192             ceros++;
193             continue;
194         }
195     }
196 }
```

```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
180 void ingresar() {
181     for (int i = 0; i < n; i++) {
182         cin >> val;
183
184         if (val == 0) {
185             ceros++;
186             continue;
187         }
188
189         totalValidos++;
190
191         if (val > mayor) mayor = val;
192         if (val < menor) menor = val;
193
194         if (val > 0 && val % 2 == 0) {
195             encolar(val);
196             pares++;
197             insertarHistorial(val, tipo: "COLA");
198         } else if (val > 0) {
199             push(val);
200             impares++;
201             insertarHistorial(val, tipo: "PILA");
202         } else {
203             insertarNegativo(val);
204             negativos++;
205         }
206     }
207 }
```

```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
180 void ingresar() {
181     for (int i = 0; i < n; i++) {
205     } else if (val > 0) {
209     } else {
210         insertarNegativo(val);
211         negativos++;
212         insertarHistorial(val, tipo: "LISTA");
213     }
214 }
215 }
216
217 void reorganizar() {
218     Nodo* aux1 = NULL;
219     Nodo* aux2 = NULL;
220
221     while (pila) {
222         Nodo* temp = pila;
223         pila = pila->siguiente;
224         temp->siguiente = aux1;
225         aux1 = temp;
226     }
227
228     while (cola.frente) {
229         Nodo* temp = cola.frente;
230         cola.frente = cola.frente->siguiente;
231     }
232 }
```

```
void reorganizar() {
    while (cola.frente) {
        temp->siguiente = aux2;
        aux2 = temp;
    }

    cout << "Lista desde pila:\n";
    mostrar(aux1);

    cout << "Lista desde cola:\n";
    mostrar(aux2);

    arbol = NULL;

    Nodo* t = aux1;
    while (t) {
        arbol = insertarBST(arbol, t->dato);
        t = t->siguiente;
    }

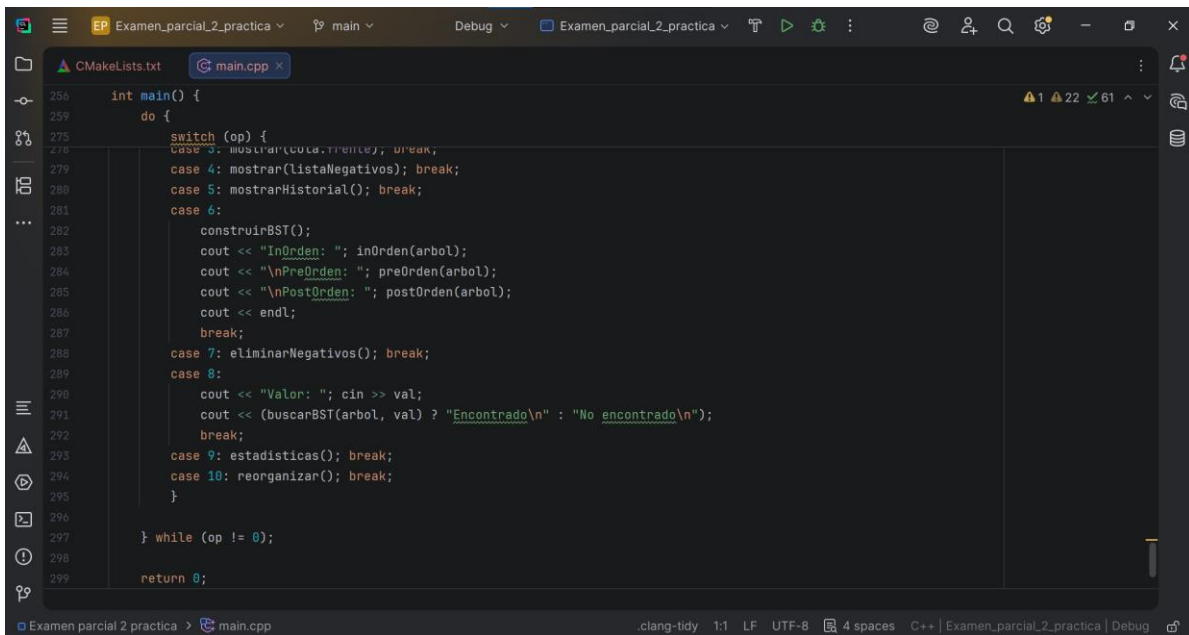
    t = aux2;
    while (t) {
        arbol = insertarBST(arbol, t->dato);
        t = t->siguiente;
    }
}
```

```
void reorganizar() {
}

int main() {
    int op, val;

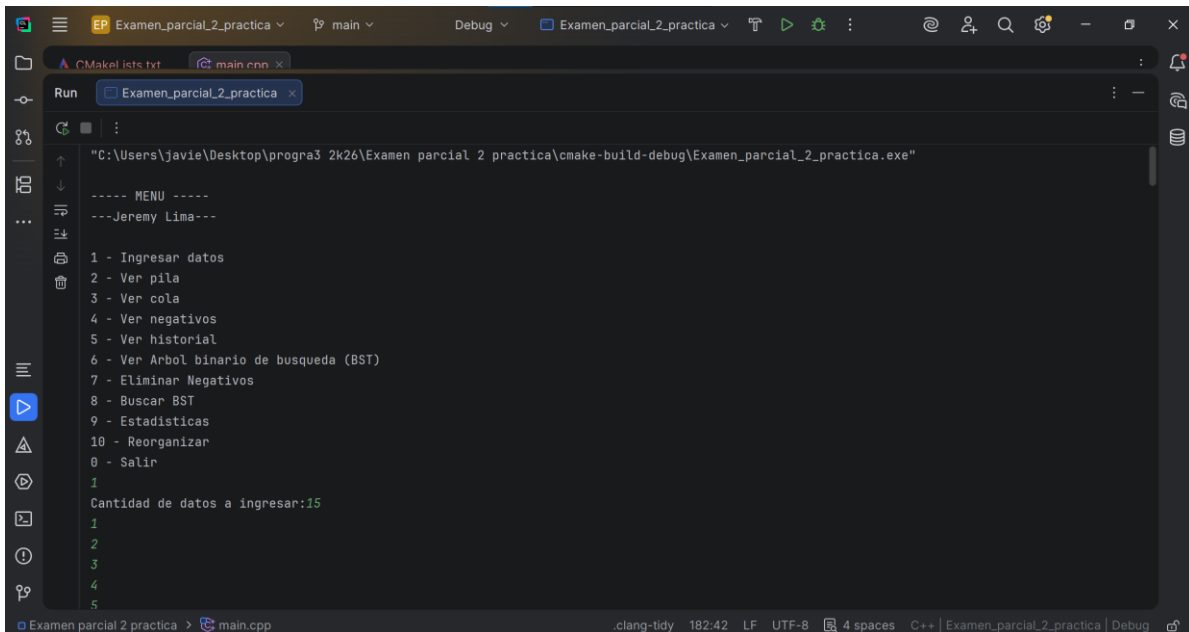
    do {
        cout << "\n---- MENU ----\n";
        cout << "---Jeremy Lima---\n\n";
        cout << "1 - Ingresar datos\n";
        cout << "2 - Ver pila\n";
        cout << "3 - Ver cola\n";
        cout << "4 - Ver negativos\n";
        cout << "5 - Ver historial\n";
        cout << "6 - Ver Arbol binario de busqueda (BST)\n";
        cout << "7 - Eliminar Negativos\n";
        cout << "8 - Buscar BST\n";
        cout << "9 - Estadisticas\n";
        cout << "10 - Reorganizar\n";
        cout << "0 - Salir\n";
        cin >> op;

        switch (op) {
            case 1: ingresar(); break;
            case 2: mostrar(pila); break;
```



```
256 int main() {
257     do {
275         switch (op) {
276             case 3: mostrar(cola.front()); break;
279             case 4: mostrar(listaNegativos); break;
280             case 5: mostrarHistorial(); break;
281             case 6:
282                 construirBST();
283                 cout << "InOrden: "; inOrden(arbol);
284                 cout << "\nPreOrden: "; preOrden(arbol);
285                 cout << "\nPostOrden: "; postOrden(arbol);
286                 cout << endl;
287                 break;
288             case 7: eliminarNegativos(); break;
289             case 8:
290                 cout << "Valor: "; cin >> val;
291                 cout << (buscarBST(arbol, val) ? "Encontrado\n" : "No encontrado\n");
292                 break;
293             case 9: estadisticas(); break;
294             case 10: reorganizar(); break;
295         }
296     } while (op != 0);
297     return 0;
299 }
```

FUNCIONAMIENTO



```
Run Examen_parcial_2_practica
"C:\Users\javie\Desktop\progra3 2k26\Examen parcial 2 practica\cmake-build-debug\Examen_parcial_2_practica.exe"

----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir
1
Cantidad de datos a ingresar:15
1
2
3
4
5
```

The screenshot shows a C++ IDE with a file named `main.cpp` open. The code displays a menu with the following options:

```
5  
6  
-1  
-2  
-3  
-4  
-5  
-6  
0  
0  
0  
  
----- MENU -----  
---Jeremy Lima---  
  
1 - Ingresar datos  
2 - Ver pila  
3 - Ver cola  
4 - Ver negativos  
5 - Ver historial  
6 - Ver Arbol binario de busqueda (BST)  
7 - Eliminar Negativos  
8 - Buscar BST
```

The status bar at the bottom indicates the compiler is `clang-tidy`, the file encoding is `UTF-8`, and the indentation is `4 spaces`.

The screenshot shows the same C++ IDE, but now with user input and program output. The code is the same as in the first screenshot, but the output window shows the following sequence of events:

```
8 - Buscar BST  
9 - Estadísticas  
10 - Reorganizar  
0 - Salir  
2  
5 3 1  
  
----- MENU -----  
---Jeremy Lima---  
  
1 - Ingresar datos  
2 - Ver pila  
3 - Ver cola  
4 - Ver negativos  
5 - Ver historial  
6 - Ver Arbol binario de busqueda (BST)  
7 - Eliminar Negativos  
8 - Buscar BST  
9 - Estadísticas  
10 - Reorganizar  
0 - Salir  
3  
2 4 6
```

The status bar at the bottom remains the same, indicating the compiler is `clang-tidy`, the file encoding is `UTF-8`, and the indentation is `4 spaces`.

```
Examen_parcial2_practica main Debug Examen_parcial2_practica
CMakeLists.txt main.cpp
Run Examen_parcial2_practica
3
2 4 6
----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir
4
-6 -5 -4 -3 -2 -1

----- MENU -----
---Jeremy Lima---
```

```
Examen_parcial2_practica main Debug Examen_parcial2_practica
CMakeLists.txt main.cpp
Run Examen_parcial2_practica
----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir
5
1) 1 -> PILA
2) 2 -> COLA
3) 3 -> PILA
4) 4 -> COLA
5) 5 -> PILA
6) 6 -> COLA
7) -1 -> LISTA
8) -2 -> LISTA
```

```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
Run Examen_parcialL2_practica
:
8) -2 -> LISTA
9) -3 -> LISTA
10) -4 -> LISTA
11) -5 -> LISTA
12) -6 -> LISTA

----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadísticas
10 - Reorganizar
0 - Salir
6

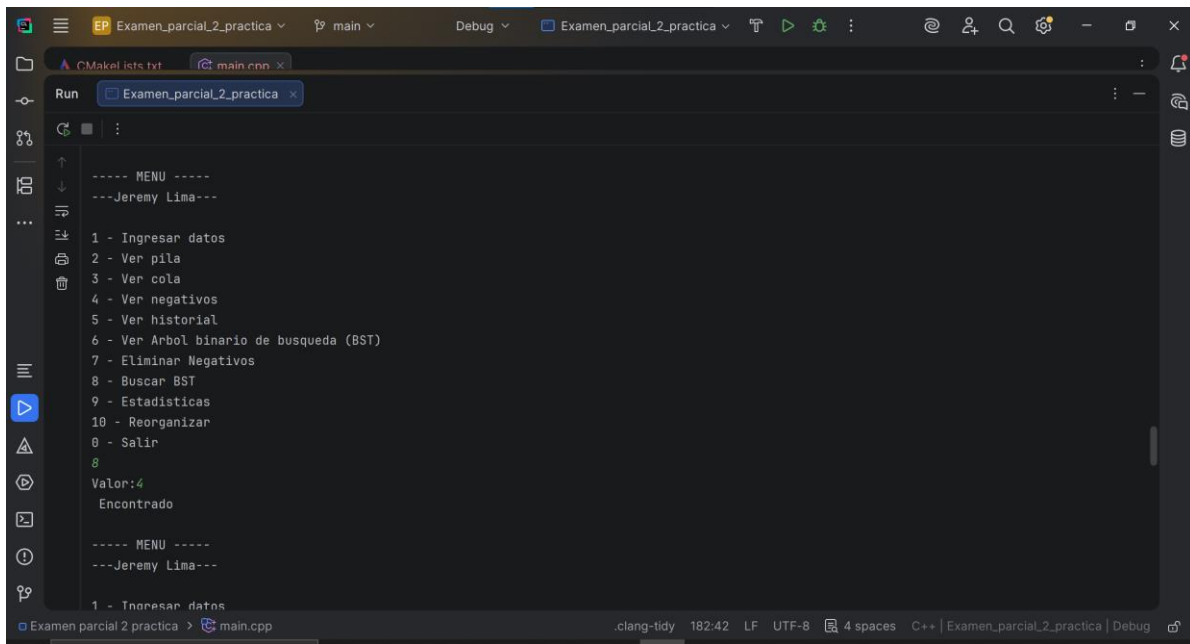
InOrden: 1 2 3 4 5 6
PreOrden: 1 2 3 4 5 6
```

```
Examen_parcialL2_practica main Debug Examen_parcialL2_practica
CMakeLists.txt main.cpp
Run Examen_parcialL2_practica
:
InOrden: 1 2 3 4 5 6
PreOrden: 1 2 3 4 5 6
PostOrden: 6 5 4 3 2 1

----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadísticas
10 - Reorganizar
0 - Salir
7
Eliminados: 6

----- MENU -----
---Jeremy Lima---
```



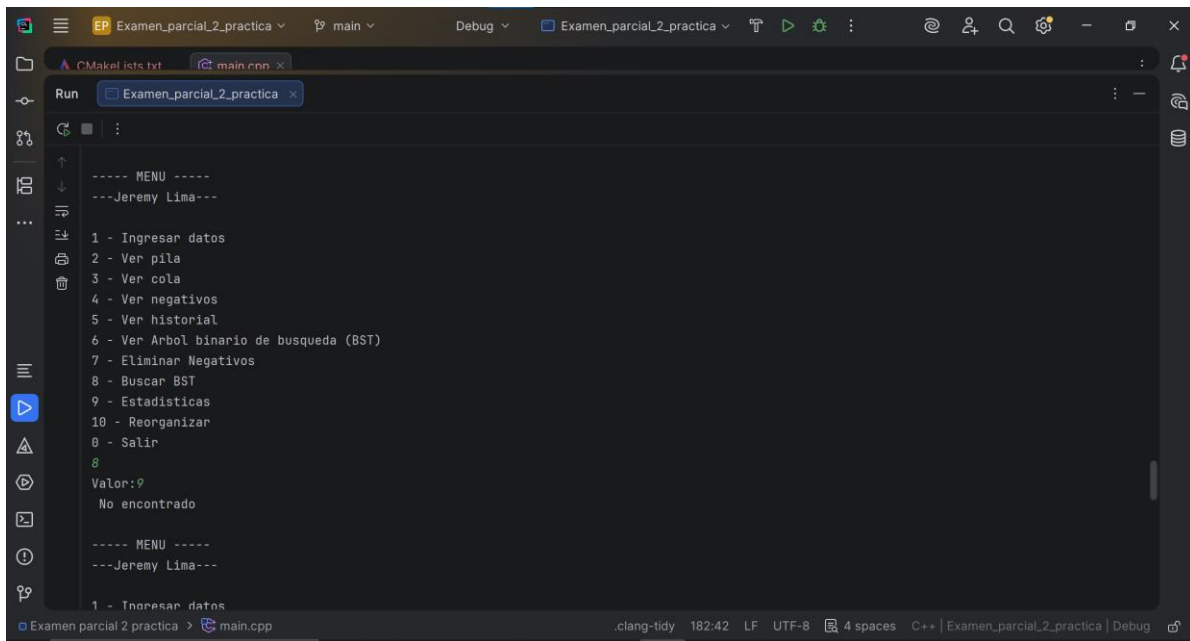
```
----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir

8
Valor:4
Encontrado

----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
```



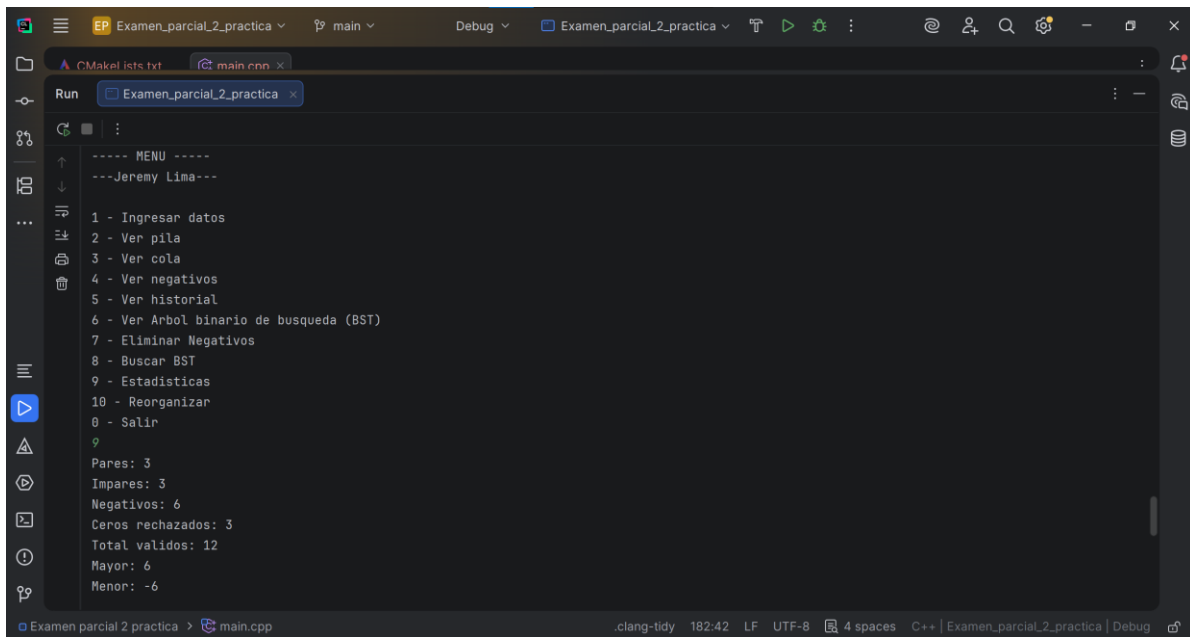
```
----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir

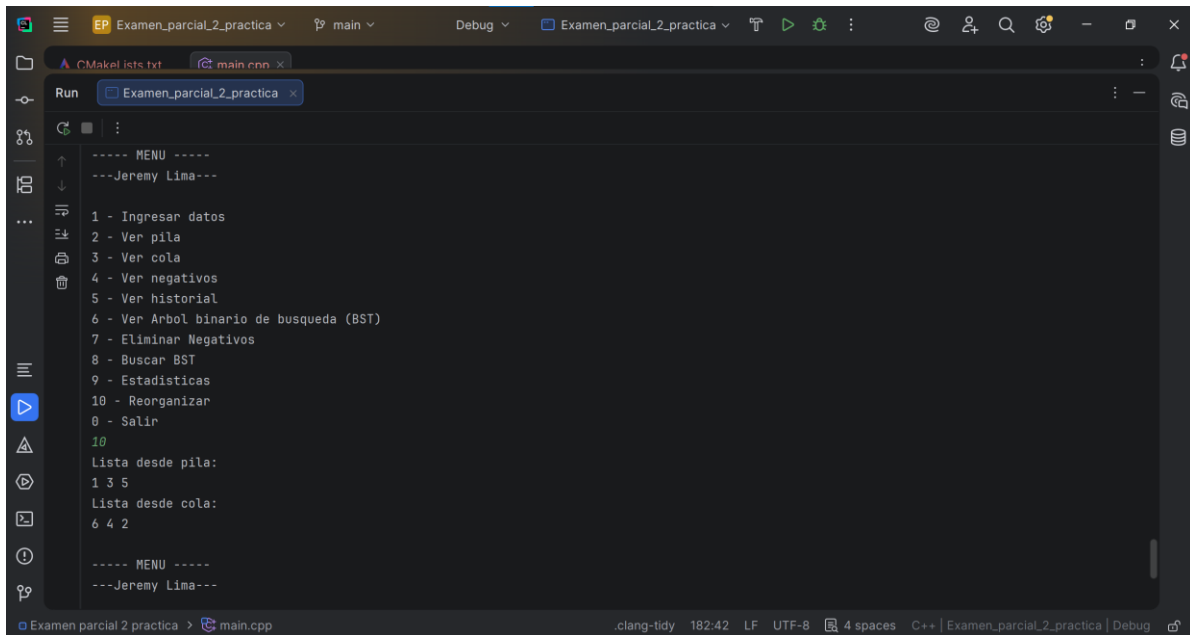
8
Valor:9
No encontrado

----- MENU -----
---Jeremy Lima---

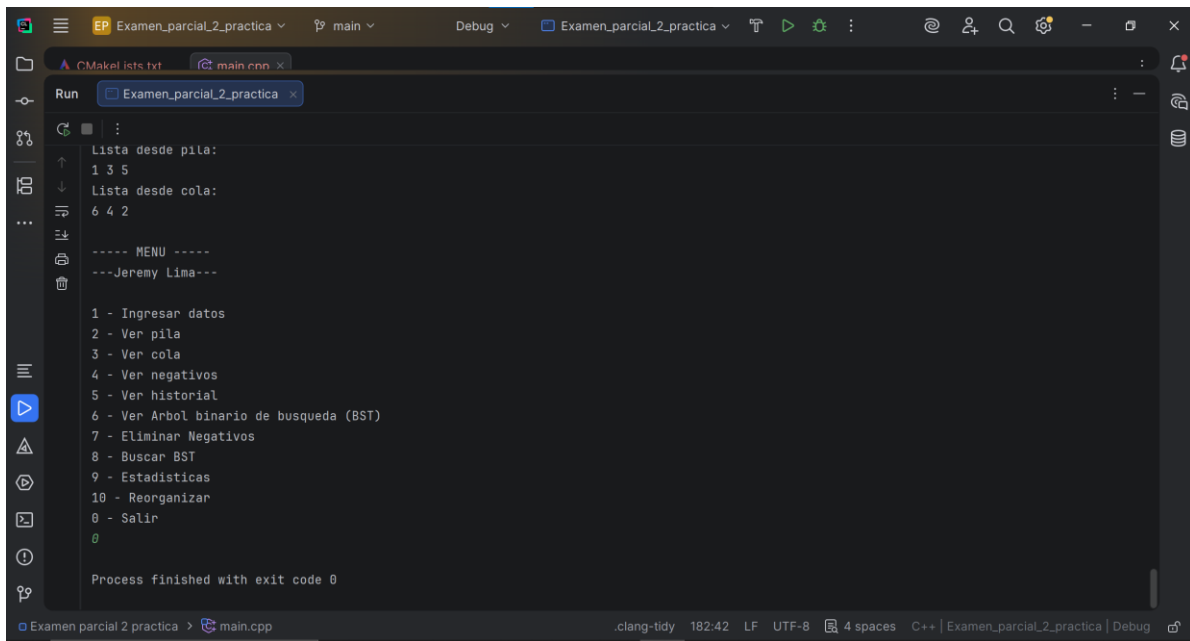
1 - Ingresar datos
```



```
----- MENU -----  
---Jeremy Lima---  
  
1 - Ingresar datos  
2 - Ver pila  
3 - Ver cola  
4 - Ver negativos  
5 - Ver historial  
6 - Ver Arbol binario de busqueda (BST)  
7 - Eliminar Negativos  
8 - Buscar BST  
9 - Estadisticas  
10 - Reorganizar  
0 - Salir  
9  
  
Pares: 3  
Impares: 3  
Negativos: 6  
Ceros rechazados: 3  
Total validos: 12  
Mayor: 6  
Menor: -6
```



```
----- MENU -----  
---Jeremy Lima---  
  
1 - Ingresar datos  
2 - Ver pila  
3 - Ver cola  
4 - Ver negativos  
5 - Ver historial  
6 - Ver Arbol binario de busqueda (BST)  
7 - Eliminar Negativos  
8 - Buscar BST  
9 - Estadisticas  
10 - Reorganizar  
0 - Salir  
10  
  
Lista desde pila:  
1 3 5  
Lista desde cola:  
6 4 2  
  
----- MENU -----  
---Jeremy Lima---
```



```
Run Examen_parcial2_practica x
:
Lista desde pila:
1 3 5
Lista desde cola:
6 4 2

----- MENU -----
---Jeremy Lima---

1 - Ingresar datos
2 - Ver pila
3 - Ver cola
4 - Ver negativos
5 - Ver historial
6 - Ver Arbol binario de busqueda (BST)
7 - Eliminar Negativos
8 - Buscar BST
9 - Estadisticas
10 - Reorganizar
0 - Salir
0

Process finished with exit code 0
```

Examen parcial 2 practica > main.cpp clang-tidy 182:42 LF UTF-8 4 spaces C++ | Examen_parcial2_practica | Debug